Project 3 — Static Type Checker

Requirements

Place `antlr-4.13.2-complete.jar` in the project root before building.

How to Compile and Run

Step 1 — Build

Generate the lexer and parser from the grammar, then compile all Java files:

```
make
```

Produces:

- `myCheckerLexer.java`
- `myCheckerParser.java`
- `myChecker.tokens`
- `myChecker.interp`
- `myCheckerLexer.interp`

Executes internally:

```
java -jar antlr-4.13.2-complete.jar -no-listener -no-visitor myChecker.g4
javac -cp .:antlr-4.13.2-complete.jar *.java
```

Step 2 — Run

Run a single test file:

```
make run FILE=test_programs/test1.c
```

Run all test programs at once:

```
make run-all
```

Clean all generated and compiled files:

```
make clean
```

Type Checking Rules

Rule 1 — Variable Declaration Before Use

Every variable must be declared before it appears in an expression or on the left-hand side of an assignment. If an undeclared identifier is used, an error is reported at the line where it appears.

```
==Error== line: Undeclared identifier.
```

Rule 2 — No Duplicate Declarations

Each identifier may only be declared once within the same scope. If the same name is declared a second time, an error is reported at the line of the duplicate declaration, regardless of whether the type is the same or different.

```
==Error== line: Redeclared identifier.
```

Rule 3 — Operand Type Consistency

Both operands of an arithmetic operator (`+`, `-`, `*`, `/`, `%`) must have the same type. Mixing `int` and `float` in a single operation is a type error. The error is reported at the line of the expression, and the result type is set to error to prevent cascading false positives.

```
==Error== line: Type mismatch for the operator + in an expression.
==Error== line: Type mismatch for the operator - in an expression.
==Error== line: Type mismatch for the operator * in an expression.
==Error== line: Type mismatch for the operator / in an expression.
==Error== line: Type mismatch for the operator % in an expression.
```

Rule 4 — Assignment Type Consistency

The type of the right-hand side expression must match the declared type of the left-hand side variable. If they differ, an error is reported even if the right-hand side already produced an expression error on the same line.

```
==Error== line: Type mismatch for the two sides of an assignment.
```

Rule 5 — Comparison Operators Produce Boolean

The operators `==`, `!=`, `>=`, `>`, `<=`, `<` always produce a result of type boolean. Both operands of the comparison must have the same type. If they differ, a type mismatch error is reported for that operator.

```
==Error== line: Type mismatch for the operator > in an expression.
```

Rule 6 — If-Statement Condition Must Be Boolean

The condition of an `if` statement must be of boolean type. A plain arithmetic expression such as a bare variable or integer constant is not boolean and will trigger an error. The error is also reported if the condition expression itself already failed type checking.

```
==Error== line: Type mismatch for condition.
```

Rule 7 — While-Loop Condition Must Be Boolean

The condition of a `while` loop must be of boolean type, by the same rule as `if`. A plain `int` or `float` expression used as a loop condition is a type error.

The error is also reported if the condition expression itself already failed type checking.

```
==Error== line: Type mismatch for condition.
```

Supported Types

Internal Code	Type	Example Literals
1	`int`	0, 42, 100
2	`float`	3.14, 0.5
3	boolean	result of `a > b`
-2	error	propagated on fail

Expected Output per Test

test1.c

```
==Error== 6: Redeclared identifier.
==Error== 8: Undeclared identifier.
==Error== 9: Type mismatch for the operator + in an expression.
==Error== 9: Type mismatch for the two sides of an assignment.
==Error== 11: Type mismatch for the two sides of an assignment.
```

test2.c

```
==Error== 14: Type mismatch for the operator % in an expression.
==Error== 14: Type mismatch for the two sides of an assignment.
==Error== 15: Type mismatch for the two sides of an assignment.
```

test3.c

```
==Error== 12: Type mismatch for condition.
==Error== 16: Type mismatch for the operator > in an expression.
==Error== 16: Type mismatch for condition.
```

test4.c

```
(no output - valid program)
```